ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН ТАЙЛАН

F.CSM302 YЙЛДЛИЙН СИСТЕМ – БИЕ ДААЛТ 1
2026–2027 оны хичээлийн жилийн намар

Сэдэв:	Multithreaded Sorting Application
Хичээл заасан багш:	Г. Ганбат /F.CS10/
Гүйцэтгэсэн оюутнууд:	Д. Цогзолдорж /B241940011/
	Э. Номинзул /B242270082/
	Ж. Ариунсанаа /B241940118/

Улаанбаатар хот
2026 он

Хураангуй

Энэхүү төслөөр бүхэл тоон массивыг хоёр хэсэгт хувааж, хэсэг бүрийг урсгалын санд эрэмбэлүүлэн, гарсан үр дүнг нэгтгэх программыг боловсруулсан. Хэрэгжилтэд Python хэл, Flask веб хүрээ, `ThreadPoolExecutor` ашигласан. Систем нь олон урсгалт Merge Sort-оос гадна найман дараалсан эрэмбэлэх алгоритм, өгөгдөл оруулах ба шалгах ажиллагаа, алхамчилсан дүрслэл, хугацааны харьцуулалтыг агуулна.

Хэрэглэгч алгоритм болон оролтын өгөгдлөө өөрчилж, эрэмбэлсэн массив, гүйцэтгэлийн хугацаа, дүрслэлийн төлвүүдийг харах боломжтой. Зөв ажиллагааны үндсэн шалгуур нь гаралтыг Python-ийн `sorted()` үр дүнтэй тэнцүүлэх явдал юм.

Тайланд төслийн онолын үндэс, архитектур, хэрэгжүүлэлт, туршилтын арга болон баримтжуулсан үр дүнг нэгтгэв.

Түлхүүр үг: олон урсгал, Merge Sort, урсгалын сан, синхрончлол, алгоритмын дүрслэл.

Гарчиг

Хураангуй **1**

1 Оршил **2**

2 Төслийн зорилго, зорилтууд **2**

3 Онолын үндэслэл **3**

4 Системийн ерөнхий зохиомж **5**

5 Системийн хэрэгжүүлэлт **6**

6 Туршилт ба үр дүн **9**

7 Хэлэлцүүлэг **11**

8 Дүгнэлт **11**

9 Ашигласан материал **11**

10 Хавсралт **13**

1 Оршил

Олон урсгалт программчлал нь нэг процессын хүрээнд ажлыг хэд хэдэн гүйцэтгэлийн урсгалд зохион байгуулах боломж олгодог. Ажлыг дэд хэсгүүдэд хуваах, тэдгээрийн дуусалтыг хүлээх, завсрын үр дүнг нэгтгэх нь үйлдлийн системийн хичээлийн чухал ойлголтууд юм. Эрэмбэлэх бодлого эдгээрийг тодорхой оролт, шалгаж болох гаралттай холбон судлахад тохиромжтой.

Operating System Concepts сурах бичгийн дөрөвдүгээр бүлгийн Multithreaded Sorting Application төсөл нь бүхэл тоон жагсаалтыг хоёр хэсэгт эрэмбэлж, нэгтгэсэн үр дүн гаргах бодлого дэвшүүлдэг [1]. Энэ ажлын хэрэгжилтэд хоёр дэд ажлыг Python-ийн урсгалын санд өгч, тэдгээрийн үр дүнг авсны дараа нэгтгэх шийдэл ашигласан.

Тайлангийн хүрээнд Python, HTML, JavaScript, CSS болон тестийн файлуудыг ашиглав. Онолын тайлбарыг сурах бичигтэй, хэрэгжүүлэлтийн тайлбарыг эх кодтой, туршилтын үр дүнг хэрэглэгчийн өгсөн дэлгэцийн зургуудтай тус тус холбов. [3, 5].

2 Төслийн зорилго, зорилтууд

2.1 Зорилго

Бүхэл тоон массивыг хэсгүүдэд хуваан олон урсгалаар эрэмбэлэх, эрэмбэлэгдсэн хэсгүүдийг нэгтгэх ажиллагааг хэрэгжүүлж, түүний үр дүн болон алгоритмын явцыг хэрэглэгчид ойлгомжтой хэлбэрээр үзүүлэх.

2.2 Зорилтууд

1. Merge Sort-ийн хуваах, рекурсээр эрэмбэлэх, нэгтгэх зарчмыг хэрэгжүүлэх.
2. Хоёр дэд ажлыг урсгалын санд өгч, дуусалтыг нь хүлээн үр дүнг нэгтгэх.
3. Оролтын өгөгдөл, алгоритмыг өөрчилж дахин ажиллуулах веб интерфэйс бүтээх.
4. Найман дараалсан алгоритмын гаралт ба хугацааг харьцуулах боломж бүрдүүлэх.
5. Алгоритмын завсрын төлвүүдийг дүрслэн, зөв ажиллагааг тест ба жишээний өгөгдлөөр шалгах.

2.3 Боловсруулах өгөгдөл

Системийн үндсэн өгөгдөл нь бүхэл тоон жагсаалт юм. Гаралт нь анхны элементүүд болон тэдгээрийн давхардлын тоог хадгалсан өсөх эрэмбэтэй жагсаалт байна. Хугацааг миллисекундээр илэрхийлнэ.

2.4 Сурах бичгийн үндсэн шаардлага

Сурах бичгийн Project 2 нь бүхэл тоон жагсаалтыг хоёр хэсэгт хувааж, хоёр эрэмбэлэх урсгалаар боловсруулах, үр дүнг гурав дахь нэгтгэх урсгалаар нэгтгэх, бүх ажил дууссаны дараа эцэг урсгалаас гаралтыг харуулах шаардлага тавьдаг [1, P-26–P-27]. Энэ нь төслийн хуваах, эрэмбэлэх, нэгтгэх зохиомжийг тодорхойлох үндсэн бодлого юм.

3 Онолын үндэслэл

3.1 Олон урсгалт гүйцэтгэлийн ойлголт

Урсгал (thread) нь программын гүйцэтгэлийн нэгж бөгөөд өөрийн таних дугаар, командын тоолуур, регистрийн төлөв, стектэй. Нэг процесст харьяалагдах урсгалууд код, өгөгдөл болон нээлттэй файл зэрэг нөөцийг хамтран ашигладаг [1]. Иймээс өгөгдөл дамжуулахад тохиромжтой боловч хамтын төлөв өөрчлөх ажиллагаанд синхрончлол шаардлагатай.

Энэ төсөлд хоёр дэд жагсаалтын эрэмбэлэлт бие биеийнхээ завсрын үр дүнгээс хамаарахгүй. Харин эцсийн нэгтгэл хоёр ажлын үр дүнг хоёуланг нь шаарддаг. Ийм хамаарлыг ажлыг салгаж өгөх, дараа нь дуусалтыг хүлээн нэгтгэх загвараар зохион байгуулсан.

3.2 Урсгал ба процессын ялгаа

Хүснэгт 1: Процесс болон урсгалын үндсэн ялгаа

Шинж	Процесс	Урсгал
Санах ойн орон зай	Тусдаа виртуал хаягийн орон зайтай.	Нэг процессын хаягийн орон зайг хуваалцана.
Гүйцэтгэлийн төлөв	Нэг буюу хэд хэдэн урсгалыг агуулна.	Өөрийн стек, регистр, командын тоолууртай.
Өгөгдөл солилцох	Процесс хоорондын харилцааны механизм ашиглана.	Хамтын өгөгдөл болон синхрончлол ашиглаж болно.
Төслийн хэрэглээ	Flask сервер программыг ажиллуулна.	Урсгалын сан дэд эрэмбэлэх ажлуудыг гүйцэтгэнэ.

Хүснэгт 1-ийн ялгаа нь эх кодод жагсаалтын зүсэлтүүдийг нэг процессын дотор функцийн параметрээр дамжуулж байгаатай холбогдоно. Төсөлд процессын сангаар ажил хуваах арга хэрэглээгүй.

3.3 Merge Sort алгоритм

Merge Sort нь хувааж эзлэх зарчимтай. Жагсаалтыг хоёр хэсэгт хувааж, хэсэг бүрийг ижил аргаар эрэмбэлээд хоёр эрэмбэлэгдсэн жагсаалтыг нэгтгэнэ. Хоосон эсвэл нэг элементтэй жагсаалт нь рекурсийн суурь тохиолдол болно.

Нэгтгэх үед хоёр жагсаалтын эхний үлдсэн элементүүдийг харьцуулж, багыг нь гаралтад нэмнэ. Аль нэг тал дуусахад нөгөө талын үлдэгдлийг залгана. Төслийн `merge()` функц тэнцүү утгад зүүн талын элементийг түрүүлж сонгох `<=` нөхцөл ашигладаг.

3.4 Merge Sort-ийн өртөг ба зөв ажиллагаа

Нэгтгэх ажиллагаа нийлбэр n элементийг нэг удаа боловсруулах тул $O(n)$ хугацаатай. Хоёр дэд бодлогын хэмжээг харгалзан рекурсийн хамаарлыг

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + O(n)$$

гэж бичнэ. Үүнээс нийт хугацаа $O(n \log n)$ болно. Жагсаалтын зүсэлт, нэгтгэх үр дүнг хадгалахад нэмэлт санах ой ашигладаг; үндсэн дараалсан алгоритмын оргил нэмэлт санах ой $O(n)$ байна.

Нэгтгэлийн алхам бүрд үлдсэн хоёр эхний элементийн багыг сонгох тул гаралтын өсөх эрэмбэ хадгалагдана. Элемент бүрийг яг нэг удаа гаралтад оруулдаг учир анхны утгууд болон давхардлын тоо хадгалагдана. Энэ шинжийг программын туршилтад `sorted(input)`-тай тэнцүүлэх замаар шалгадаг.

3.5 Зэрэгцээ эрэмбэлэлтийн зарчим

Хоёр дэд хэсгийг тус тусдаа ажил болгон өгөх нь нэг хэсгийн эрэмбэлэлтийг нөгөө хэсгийн үр дүнгээс хамааралгүй эхлүүлэх боломж олгоно. Ажиллах хугацааг ойлголтын түвшинд

$$T_{par}(n) \approx \max(T_L, T_R) + T_{merge} + T_{overhead}$$

гэж тайлбарлаж болно. Энд T_L, T_R нь дэд ажлуудын хугацаа, T_{merge} нь эцсийн нэгтгэл, $T_{overhead}$ нь урсгалын сан үүсгэх, ажил өгөх, үр дүн хүлээхтэй холбоотой зардал юм. Энэ томьёо нь дэд ажлууд зэрэг гүйцэтгэгдэх боломжтой үеийн загвар бөгөөд бодит хурдатгалыг орлох хэмжилт биш.

Төсөл зөвхөн эхний хуваалтын хоёр ажлыг урсгалын санд өгдөг. Дэд ажлын доторх рекурсив Merge Sort нь дараалсан байдлаар ажиллана. Иймээс рекурсийн бүх түвшинд шинэ урсгал үүсгэдэг бүтэц биш болно.

3.6 ThreadPoolExecutor-ийн ажиллагаа

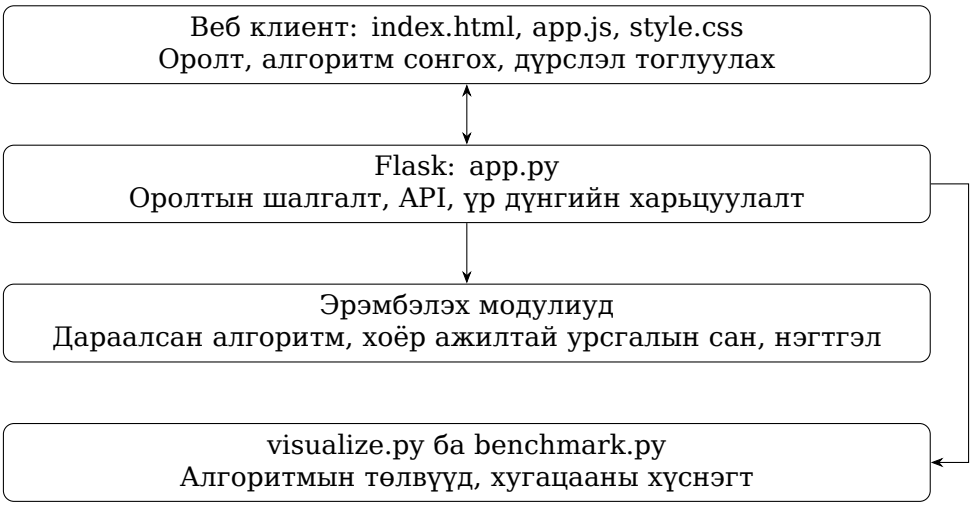
Урсгалын сан нь ажлыг worker урсгалуудаар гүйцэтгэх загвар бөгөөд сангийн хэмжээ нь нэгэн зэрэг ажиллах worker-ийн дээд хязгаарыг тогтооно. Сурах бичигт урсгалын санг урсгалын нөөц ба ажлын гүйцэтгэлийг зохион байгуулах арга гэж авч үздэг [1].

Код дахь `ThreadPoolExecutor(max_workers=2)` нь хоёр хүртэл worker ашиглана. `submit()` функц дэд ажлыг санд өгч `Future` объект буцаана. `Future.result()` нь ажил дуусаагүй бол хүлээгээд үр дүнг авна. Хоёр үр дүн бэлэн болсны дараа дуудсан урсгал `merge()`-ээр эцсийн жагсаалтыг үүсгэнэ. Сангийн worker-ийн дээд хэмжээ нь тухайн гүйлтэд оролцсон ялгаатай урсгалын тоог өөрөө хэмжсэн үзүүлэлт биш.

4 Системийн ерөнхий зохиомж

4.1 Системийн архитектур

Систем нь веб клиент, Flask сервер, эрэмбэлэх модулиуд гэсэн үндсэн хэсгүүдээс бүрдэнэ. Клиент оролтыг илгээж, сервер тооцоолол ба шалгалтыг хийж, үр дүн болон дүрслэх төлвүүдийг буцаана. Зураг 1-т кодын модуль ба дуудлагын холбоог үзүүлэв.



Зураг 1: Эх кодод тулгуурласан системийн архитектур. Сервер эрэмбэлэлт, дүрслэлийн өгөгдөл болон benchmark-ийг тус тусын функцээр боловсруулна.

4.2 Програмын бүтэц

Хүснэгт 2: Үндсэн файл, хавтасны үүрэг

Файл, хавтас	Хариуцах ажиллагаа
`app.py`	Flask замууд, хүсэлт задлах, үр дүн ба хугацаа буцаах.
`src/sorting/merge_sort.py`	Рекурсив Merge Sort болон хоёр жагсаалт нэгтгэх.
`src/sorting/parallel_sort.py`	Хоёр дэд ажил, Future, trace үйл явдал.
`src/sorting/visualize.py`	Найман алгоритмын завсрын төлөв үүсгэх.
`src/sorting/benchmark.py`	Хэмжээ бүрийн хугацааны медиан тооцох.
`templates/`, `static/`	HTML загвар, JavaScript удирдлага, CSS загварчлал.
`tests/`	Алгоритм, дүрслэл, API-ийн автомат шалгалтууд.

4.3 Модуль хоорондын ажиллагаа

`prepare_request()` оролт болон алгоритмыг сонгож, `time_algorithms()` дараалсан ба олон урсгалт үр дүнг тооцно. Stream замд `visualized_sort()` тусдаа гүйлтээр дүрслэх төлвүүд үүсгэнэ. Сервер эдгээрийг `Queue`-д байршуулж, NDJSON хэлбэрээр клиентэд дамжуулдаг.

5 Системийн хэрэгжүүлэлт

5.1 Өгөгдөл оруулах, шалгах

Хэрэглэгч бүхэл тоонуудыг интерфэйсийн талбарт таслалаар тусгаарлан оруулна. Серверийн `parse_numbers()` мөр болон жагсаалт хэлбэрийн оролтыг боловсруулдаг. Мөрөн оролтын таслал, цэгтэй таслалыг зайгаар сольж, элементүүдийг бүхэл тоонд хувиргана. Хэмжээ, тоон төрөл, утгын абсолют хязгаарыг шалгасны дараа алгоритмд дамжуулна.

API дор хаяж дөрвөн элемент авна. Утгын абсолют хэмжээ $2^{53} - 1$-ээс ихгүй байна. Алгоритмын хэмжээний хязгаарыг Хүснэгт 3-т үзүүлэв. Counting Sort-ийн хувьд утгын мужид нэмэлт шалгалт хийнэ. Шалгалтаар татгалзсан хүсэлтэд HTTP 400 болон Монгол хэлээр тайлбарласан мэдэгдэл буцаадаг.

Хүснэгт 3: Алгоритм сонгох хүсэлтийн элементийн дээд хэмжээ

Алгоритм	Дээд хэмжээ
Merge Sort, Heap Sort, Counting Sort	5,000
Quick Sort, Shell Sort	2,000
Bubble Sort, Insertion Sort, Selection Sort	1,000

5.2 Массивыг хэсгүүдэд хуваах

`middle = len(numbers) // 2` заагаар зүүн `numbers[:middle]` ба баруун `numbers[middle:]` жагсаалтыг үүсгэнэ. Сондгой урттай үед баруун хэсэг нэг элементээр урт байна. Хэсэг бүр тусдаа жагсаалт тул үндсэн эрэмбэлэлтэд нэг массивын ижил нүдийг зэрэг өөрчлөх шаардлагагүй.

5.3 Олон урсгалаар эрэмбэлэх

Хугацаа хэмжих замд `trace=False` тохиргоо ашиглаж, завсрын төлвийн хадгалалтын зардлыг оруулахгүй. Дараах нь `_parallel_merge_sort_plain()` функцийн гол хэсэг юм [3].

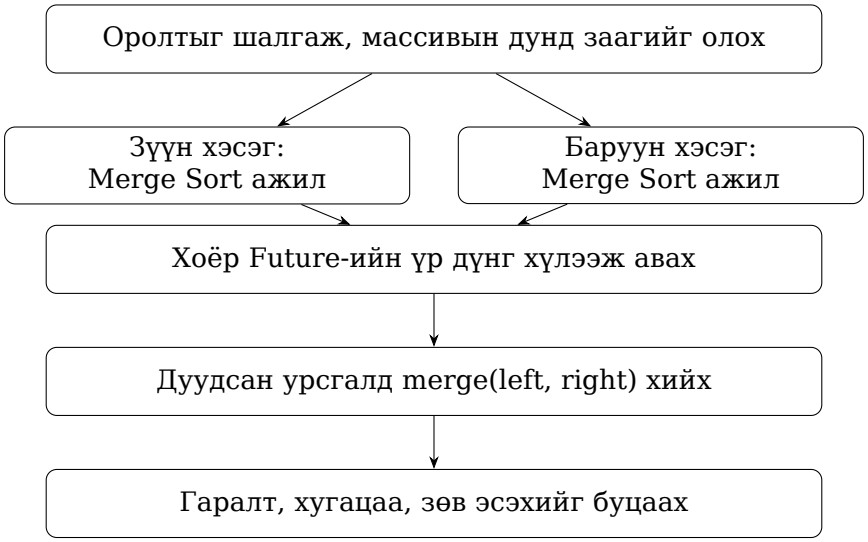
```
with ThreadPoolExecutor(
  max_workers=2, thread_name_prefix="SortWorker"
) as executor:
  left_future = executor.submit(merge_sort, numbers[:middle])
  right_future = executor.submit(merge_sort, numbers[middle:])
  left_sorted = left_future.result()
  right_sorted = right_future.result()
return merge(left_sorted, right_sorted)
```

5.4 Эрэмбэлэгдсэн хэсгүүдийг нэгтгэх

`merge()` функц i, j индексээр хоёр жагсаалтыг уншиж, бага элементийг шинэ жагсаалтад нэмнэ. Үлдэгдлийг `extend()`-ээр залгасны дараа эцсийн массив буцна. Хоёр `result()` дуудлага нь нэгтгэлээс өмнөх хамаарлыг хангаж өгдөг.

5.5 Өгөгдөл боловсруулах дараалал

Зураг 2 нь үндсэн тооцооллын хуваах, эрэмбэлэх, хүлээх, нэгтгэх дарааллыг харуулна. Дүрслэлийн төлвүүд энэ хугацаа хэмжих гүйлтийн дараа тусдаа үүсдэг.



Зураг 2: Олон урсгалт Merge Sort-ийн өгөгдөл боловсруулах дараалал. Хоёр дэд ажил ижил урсгалын санд өгөгдөнө.

5.6 Бусад эрэмбэлэх алгоритмууд

Найман алгоритмыг ижил оролт дээр ажиллуулах боломж нь аргуудын ажиллагаа, хугацааны ялгааг судлахад зориулагдсан. Сонгосон дараалсан алгоритмаас үл хамааран олон урсгалт харьцуулах тал нь Merge Sort байна.

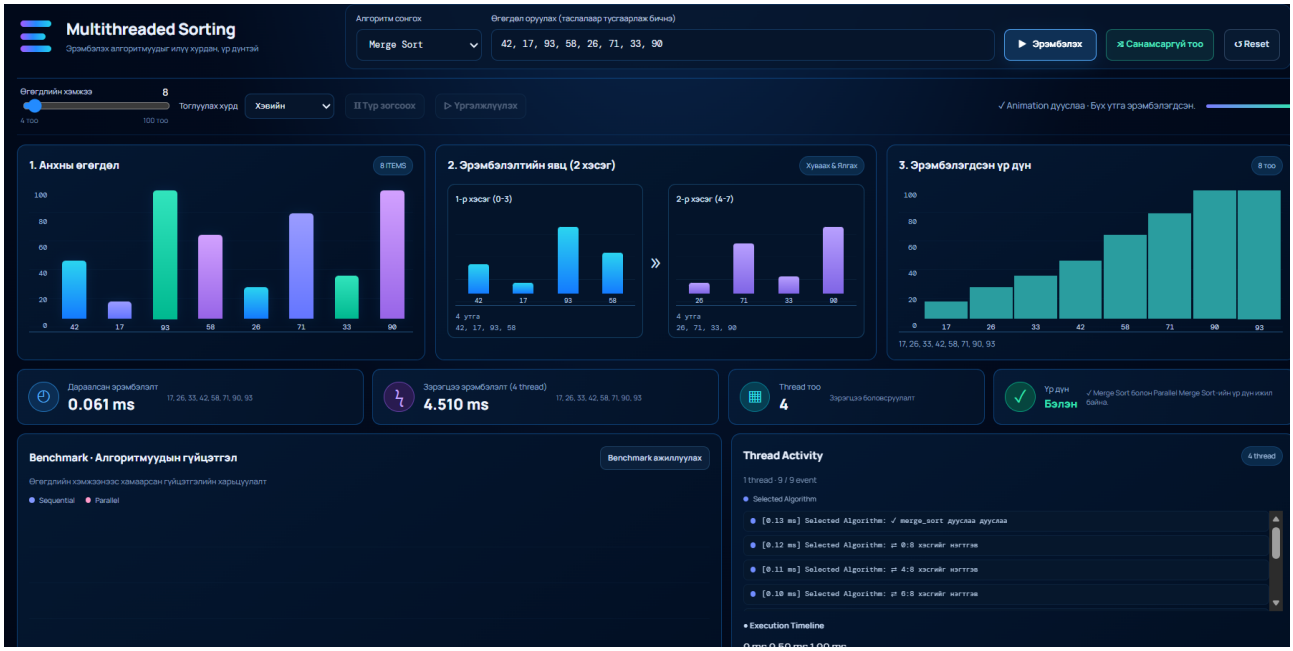
Хүснэгт 4: Хэрэгжүүлсэн дараалсан алгоритмуудын үндсэн арга

Алгоритм	Кодод ашигласан арга
Merge Sort	Рекурсив хуваалт, эрэмбэлэгдсэн хэсгүүдийн нэгтгэл.
Bubble Sort	Хөрш элементүүдийг сольж, солилцоогүй үед эрт дуусгана.
Insertion Sort	Дараагийн элементийг эрэмбэлэгдсэн хэсэгт оруулна.
Selection Sort	Үлдсэн хэсгийн хамгийн бага утгыг олж байрлуулна.
Quick Sort	Дунд байрлалын pivot, хоёр талаас ойртох индексээр хуваана.
Heap Sort	Max-heap үүсгээд их утгыг төгсгөлд шилжүүлнэ.
Shell Sort	$n/2$-оос эхлэх бүхэл завсрыг хоёр дахин бууруулна.
Counting Sort	Хамгийн бага утгаар шилжүүлэн давтамж тоолж жагсаалт үүсгэнэ.

Counting Sort-д $k = \max(x) - \min(x) + 1$ хэмжээтэй давтамжийн массив хэрэглэнэ. Түүний хугацаа $O(n + k)$; код $k > \max(10^6, 100n)$ үед оролтыг хүлээн авахгүй. Ингэснээр давтамжийн массивын хэмжээг шалгадаг.

5.7 Хэрэглэгчийн интерфэйс

Интерфэйсийн дээд хэсэгт алгоритм сонгох жагсаалт, оролтын талбар, эрэмбэлэх, санамсаргүй тоо үүсгэх, дахин эхлүүлэх удирдлагууд байрлана. Өгөгдлийн хэмжээний гулсуур 4–100 элементийн жишээ үүсгэнэ. Үндсэн гурван самбар анхны өгөгдөл, хоёр хэсгийн төлөв, эцсийн үр дүнг үзүүлдэг.



Зураг 3: Merge Sort-ийн ажиллагааны дэлгэц. Оролтын найман утга, хоёр хэсэг, өсөх дараалалтай гаралт, хугацаа болон үйл явдлын жагсаалт харагдана. Хэрэглэгчийн өгсөн зураг.

Зураг 3-т $[42, 17, 93, 58, 26, 71, 33, 90]$ массивыг $[17, 26, 33, 42, 58, 71, 90, 93]$ болгон эрэмбэлсэн байна. Үр дүнгийн самбар хоёр аргын гаралт ижил гэж мэдээлж, дүрслэлийн тоглуулалт дууссан төлөв харагдана.

5.8 Үр дүнгийн дүрслэл

`visualized_sort()` эхний гүйлтээр үйлдлийн тоог олж, хоёр дахь гүйлтээр тодорхой алхам тутмын массивын төлөвийг хадгална. Хадгалах алхмын зай:

$$\text{stride} = \max\left(1, \left\lceil \frac{\text{total_steps}}{\text{event_budget}(n)} \right\rceil\right).$$

Эхлэх, дуусах төлөвийг нэмж хадгалдаг. Клиент төлвүүдийг багана, үйл явдлын жагсаалт, хугацааны зурвасаар үзүүлнэ. Түр зогсоох, үргэлжлүүлэх, хурд өөрчлөх нь дүрслэлийн тоглуулалтад үйлчилдэг.

Stream API нь `Selected Algorithm` шошготой дүрслэх гүйлтийн төлвүүдийг дамжуулна. Зураг дахь 9 үйл явдал энэ дараалалд хамаарна. Урсгалын сангийн тохиргоо нь эх кодын `max_workers=2`; интерфэйсийн тогтмол “4 thread” шошгыг хэмжсэн урсгалын тоо гэж ашиглаагүй.

6 Туршилт ба үр дүн

6.1 Туршилтын орчин ба мэдээллийн эх үүсвэр

Энэ бүлгийн бодит үр дүн нь хэрэглэгчийн өгсөн Merge Sort, Bubble Sort, Selection Sort-ийн гурван дэлгэцийн зурагт тулгуурлана [5]. Зургууд веб интерфэйсийн гүйлт, гаралт, хугацааг харуулна. Тухайн компьютерийн CPU, RAM болон runtime-ийн хувилбар зурагт байхгүй тул хугацааг тэр орчны нэг удаагийн ажиглалт гэж авч үзэв.

Хүснэгт 5: Туршилтын бүртгэлийн хүрээ

Талбар	Эх материалаар тогтоосон мэдээлэл
Хэрэгжилтийн хэл	Python; серверийн кодоор тогтоосон.
Веб хүрээ	Flask; `app.py`-д ашигласан.
Клиент	HTML, CSS, JavaScript бүхий веб интерфэйс.
Үр дүнгийн баримт	Гурван алгоритмын эцсийн төлөвийн screenshot.
Хугацааны нэгж	Серверийн хэмжилтийг харуулсан миллисекунд.

6.2 Туршилтын өгөгдөл, арга

Үр дүнг оролт, хүлээгдэх гаралт, дэлгэц дээрх бодит гаралт гэсэн дараалаар тэмдэглэв. Merge Sort ба Bubble Sort-д ижил найман утга, Selection Sort-д хоёр удаа давтагдах 7 утгатай массив ашигласан байна.

Хүснэгт 6: Дэлгэцийн зургаар баримтжуулсан туршилтын тохиолдлууд

Код	Алгоритм	Оролт ба шалгах зүйл
T01	Merge Sort	42, 17, 93, 58, 26, 71, 33, 90; өсөх эрэмбэ, хоёр гаралтын тэнцэл.
T02	Bubble Sort	T01-тэй ижил өгөгдөл; алгоритм солиход ижил эцсийн дараалал гарах.
T03	Selection Sort	25, 7, 60, 7, 67, 30, 82, 62; давхардсан утгын тоо хадгалагдах.

Програмын үндсэн шалгуур нь $R_{seq} = R_{par} = \text{sorted}(A)$ юм. Stream замд дүрслэлийн эцсийн массивыг мөн харьцуулдаг. Гурван зурагт харагдах гаралтыг энэ өсөх дараалал болон элементүүдийн тоотой тулган тайлбарлав. Эх кодын автомат тестүүд алгоритм, дүрслэл, API-ийн нэмэлт тохиолдлуудыг агуулна [3].

6.3 Гүйцэтгэлийн хэмжилтийн арга

`time_algorithms()` нь `perf_counter()`-ийн эхлэл, төгсгөлийн зөрүүг 1,000-аар үржүүлэн миллисекунд тооцдог. Эхлээд сонгосон дараалсан алгоритм, дараа нь `trace=False` олон урсгалт Merge Sort ажиллана. Урсгалын сан үүсгэх, үр дүн хүлээх, эцсийн нэгтгэх хугацаа олон урсгалт хэмжилтэд орно. Клиентийн хөдөлгөөнт дүрслэлийн хугацаа орохгүй.

`run_benchmark()` нь `Random(42)` ашиглан $[-10000, 10000]$ мужийн 100, 250, 500, 1,000 элементтэй өгөгдөл үүсгэдэг. Хэмжээ бүрд алгоритм бүр ижил массивын хуулбарыг авч, анхдагч гурван давталтын медианыг буцаана. Энэ нь тогтмол үүсгэгчтэй хэмжилтийн горим юм.

6.4 Эрэмбэлэлтийн үр дүн

Зураг 4-т Bubble Sort нь T01-тэй ижил гаралт өгчээ. Зураг 5-т $[7, 7, 25, 30, 60, 62, 67, 82]$ гаралт харагдаж, 7 утгын хоёр давталт хадгалагдсан.



Зураг 4: T02: Bubble Sort-ийн эцсийн үр дүн. Найман утга өсөх эрэмбэд орж, олон урсгалт Merge Sort-ийн гаралттай ижил.



Зураг 5: T03: Selection Sort-ийн давхардсан утгатай туршилт. Гаралтад хоёр ширхэг 7 утга хадгалагдав.

6.5 Гүйцэтгэлийн хэмжилтийн үр дүн

Хүснэгт 7: Гурван screenshot дээрх нэг удаагийн хугацааны утгууд, мс

Тест	Дараалсан алгоритм	Дараалсан	Олон урсгалт
T01	Merge Sort	0.061	4.510
T02	Bubble Sort	0.030	3.588
T03	Selection Sort	0.025	3.205

Олон урсгалт багана бүх мөрд Parallel Merge Sort-ийн хугацаа юм. Эдгээр нь медиан бус, тус тусын гүйлтийн утга. Найман элементтэй гурван тохиолдолд олон урсгалт хугацаа их гарсан. Өөр ороллтой T03 болон нэг удаагийн T01, T02 хэмжилтээр алгоритмуудын ерөнхий хурдыг эрэмбэлэхгүй. Зурагт benchmark график хоосон тул түүнийг хугацааны нэмэлт үр дүн гэж тооцоогүй.

7 Хэлэлцүүлэг

7.1 Хэрэгжүүлэлтийн үр дүнгийн тайлбар

Төслийн гол шийдэл нь хоёр дэд ажлын үр дүнгээс хамаарах нэгтгэлийг `Future.result()`-ээр зохион байгуулсан явдал юм. Ажил бүр өөрийн жагсаалтын зүсэлтийг боловсруулж, эрэмбэлсэн шинэ жагсаалт буцаана. Энэ загвар нь хуваах, ажлын дуусалтыг хүлээх, үр дүн нэгтгэх ойлголтуудыг кодын тодорхой ажиллагаатай холбодог.

Интерфэйсийн дүрслэл алгоритмын алхмыг судлахад, хугацааны хэмжилт тооцооллын өртгийг харьцуулахад чиглэнэ. Төлвүүдийг тусдаа гүйлтээр үүсгэдэг тул тоглуулах хурд серверийн хэмжилтэд нөлөөлөхгүй.

7.2 Гүйцэтгэлийн үр дүнгийн утга

Хүснэгт 7-ийн ажиглалт нь урсгал ашигласан байдал болон хурдатгалыг тусад нь үнэлэх үндэслэл болно. Жижиг оролтод сан үүсгэх, ажил өгөх, хүлээх зардал нийт хугацаанд нөлөөлөх боломжтой. Бүртгэгдсэн туршилт эдгээрийг тус бүрээр хэмжээгүй тул хугацааны ялгааг нэг шалтгаанд хамааруулахгүй.

Хүснэгтийн хугацаа нь хэрэглэгчийн өгсөн зургууд дахь нэг удаагийн гүйлтийн ажиглалт юм. Өөр компьютер, утгын тархалт, хэмжээтэй оролтод хугацаа өөрчлөгдөнө. Тиймээс бүх нөхцөлд нэг алгоритм давамгайлна гэж ерөнхийлөхгүй; харин сонгосон өгөгдөлд алгоритмын арга болон ажлын зохион байгуулалтын нөлөөг тайлбарлана.

8 Дүгнэлт

Multithreaded Sorting Application төсөлд бүхэл тоон жагсаалтыг хоёр хэсэгт хуваах, хэсэг бүрийг урсгалын санд Merge Sort-оор эрэмбэлэх, хоёр үр дүнг хүлээн нэгтгэх ажиллагааг хэрэгжүүлсэн. Үүнийг Flask API, оролтын шалгалт, найман дараалсан алгоритм, алгоритмын төлвийн дүрслэл болон хугацааны харьцуулалттай холбосон.

Өгсөн гурван дэлгэцийн зурагт оролтын найман элемент өсөх дарааллаар гарч, хоёр аргын үр дүн ижил байв. Selection Sort-ийн жишээнд давхардсан 7 утга хадгалагдсан. Нэг удаагийн хэмжилтүүдэд олон урсгалт Merge Sort-ийн хугацаа их байсан тул хурдатгалтай гэсэн дүгнэлт хийгээгүй.

Төслийн хүрээнд олон урсгалт ажлыг хуваарилах, дуусалтыг хүлээх, үр дүн нэгтгэх болон алгоритмын ажиллагааг дүрслэх ойлголтуудыг нэг программд уялдуулав.

9 Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*. 10th edition. Бүлэг 4, §4.1, §4.5.1; Programming Projects, Project 2, P-26–P-27.

[2] F.CSM302 Үйлдлийн систем. *Бие даалт 1-ийн удирдамж*. `OS_BD1_Guide.pdf`, 2–4-р хуудас.

[3] Төслийн эх код. `F.CSM302-TeamProject-4-2-6-main.zip`: `app.py`, `src/sorting/`, `tests/`, `templates/`, `static/`.

[4] Python Software Foundation. *Python 3 Documentation*. Олон урсгалт программчлал, урсгалын сан, дараалал болон автомат тестийн баримт бичиг. `https://docs.python.org/3/`

[5] Pallets Projects. *Flask Documentation*. Веб сервер, хүсэлт боловсруулах болон API хэрэгжүүлэх баримт бичиг. `https://flask.palletsprojects.com/`

[6] MDN Web Docs. *HTML, CSS, JavaScript болон Canvas API*. Веб интерфэйс, үйлдлийн удирдлага, график дүрслэлийн баримт бичиг. https://developer.mozilla.org/

[7] The LaTeX Project. *LaTeX Documentation*. Тайлангийн бүтэц, зураг, хүснэгт болон PDF баримт боловсруулах баримт бичиг. https://www.latex-project.org/help/documentation/

10 Хавсралт

10.1 А. Программыг ажиллуулах дараалал

Доорх дараалал нь өгсөн Flask эх кодыг ажиллуулахад зориулагдсан. Python орчинд Flask суулгаад төслийн `app.py` байгаа хавтсаас серверийг эхлүүлнэ. Эх тайлангийн хэмжилтэд ашигласан Flask хувилбарыг жишээ командод тусгав.

```
python3 -m pip install Flask==3.1.3
python3 -m flask --app app run
```

Браузераар `http://127.0.0.1:5000` хаягийг нээнэ. Алгоритмаар Merge Sort сонгож, номын жишээний арван тоог оруулаад эрэмбэлэх үйлдлийг эхлүүлнэ. Хүлээгдэх гаралт: 2, 3, 4, 6, 7, 8, 12, 15, 18, 19. Өөр алгоритм сонгож ижил өгөгдлийн гаралтыг харьцуулан үзэж болно.

10.2 Б. API-ийн үндсэн замууд

Хүснэгт 8: Серверийн API ба хариуны зориулалт

Зам	Арга	Зориулалт
`/`	GET	Үндсэн веб интерфэйс.
`/api/sort`	POST	Оролт, хоёр гаралт, хугацаа, зөв эсэх.
`/api/sort/stream`	POST	NDJSON үйл явдлууд ба эцсийн үр дүн.
`/api/benchmark`	GET, POST	Үүсгэгчийн өгөгдөл дээрх алгоритмуудын медиан хугацаа.

Хүсэлтийн жишээ:

```
{
  "algorithm": "merge_sort",
  "numbers": [7, 12, 19, 3, 18, 4, 2, 6, 15, 8]
}
```

10.3 В. Тестийн файлуудыг ажиллуулах

Төслийн үндсэн хавтсаас дараах тушаалаар өгсөн тестүүдийг дуудаж болно.

```
python3 -m unittest discover -v
```

`test_sorting.py` нь эрэмбэлэх функцүүдийг, `test_visualize.py` нь дүрслэх алгоритмын гаралт ба үйл явдлыг, `test_app.py` нь веб API-ийн хариу болон оролтын шалгалтыг хамарна. Энэ тушаал нь эх кодод буй тестийг давтан ажиллуулах заавар бөгөөд шинэ гүйлтийн амжилтын мэдэгдэл биш болно.